

C++20

多策略组合优化算法移植与性能分析报告

Python algo.py 兼容实现、凸优化重构与性能评估

提交对象：面试技术评审

项目类型：8 个时间序列的多策略组合优化

实现环境：C++20 / MSVC x64 / Windows

报告依据：algo.py、C++ 源码、项目文档及实际测试记录

摘要

本项目将原始 Python 程序 algo.py 的 8 策略组合优化流程迁移到 C++20，并保留两条相互独立的求解路径。Python Compatible 是主要交付路径，目标是保持原程序的数据处理、统计口径、约束、SLSQP 调用方式和输出行为；High Accuracy 是在此基础上的扩展，用于把可以明确表示为凸 LP/QP 的部分交给 HiGHS，并增加协方差正则化、PSD 修复和 KKT 验收。两条路径的目标不同，不能用增强路径替代兼容结果。

在当前 data.xls 上，C++ Compat 输出与项目保存的 algo.py 基准快照在四组均值-方差策略的显示精度内基本一致；最大分散/等权存在较明显差异，主要差异为 Calmar 5.531 和 Sharpe 0.363。C++ 完整进程重复运行中位数为 Compat 350.57 ms、HighAccuracy 401.23 ms；严格重启后的单次进程耗时分别为 940.586 ms 和 403.088 ms，均未超过 1 秒，但冷启动受系统调度、DLL 装载和磁盘状态影响明显。

报告中的误差、时间和测试结论均来自项目已有记录。Python 运行环境当前未安装 pandas，因此本报告不把一次重新执行 Python 的结果作为证据；对比表采用仓库保存的 reference.txt 基准快照。

1. 项目概述

题目要求将包含 8 个时间序列 a~h 的 XLS 数据交由 C++ 实现，输出尽可能接近 algo.py，并将完整程序执行时间控制在 1 秒以内。同时需要说明优化模型、求解过程、执行性能、数值稳定性和工程风险。

本实现的处理链为：

XLS → 累计 PnL 转日增量 → 3σ 截尾预处理 → 60 日滚动窗口 → 普通/指数加权统计量 → 权重优化 → 组合收益 → EvaluatePNL 指标。

当前配置为 window=60、keep=15、yr_dates=250。程序每 15 个交易日重新计算一次权重，其余日期沿用上一期权重。默认入口为 PythonCompatible；使用 --high 或 --high-accuracy 才启用 HighAccuracy。

2. 原 Python 算法分析

algo.py 先读取第一个工作表，将 Date 作为索引，将 a~h 作为策略序列。df.diff(1).fillna(0) 把累计 PnL 转为日增量。norm_preprocess 对每个时点使用此前 window 个已经处理后的样本计算均值和样本标准差，对超过 $\text{mean} \pm 3\sigma$ 的当前观测做截尾。

滚动窗口在调仓日取过去 60 行，并按 keep=15 做 rolling(keep).sum().dropna()。窗口不包含当前调仓日，避免当前结果进入当天权重决策。权重向量 w 表示 8 个策略资金占比，原 Python 约束为：

$$\sum w_i = 1, \quad w_i \geq 0, \quad \sum |w_i - w_{i-1}| \leq 0.1.$$

此外还包含四个分组的上下限约束：a~d 之和、e、f、g~h 之和均位于 0.15 与 0.35 之间。

3. 优化模型

当 risk_averse 非空时，原程序使用均值-方差目标：

$$\min 1/2 \times \lambda \times w^T \Sigma w - \mu^T w$$

其中 w 为权重, μ 为滚动样本均值, Σ 为协方差矩阵, λ 为风险厌恶系数。 $\mu^T w$ 表示组合期望收益, $w^T \Sigma w$ 表示组合方差。 $\lambda=0$ 时模型退化为 $\max \mu^T w$ 的线性问题。

当 `risk_averse` 为空时, 原程序最大化分散比率:

$$\max (v^T w) / \sqrt{w^T \Sigma w}$$

v 为各策略波动率向量。该目标含平方根和分式, 且与 L1 换手约束组合后在边界处存在不可导点, 是 `Compat` 路径最敏感的部分。

4. 凸性分析

均值-方差目标的 Hessian 为 $\nabla^2 f(w) = \lambda \Sigma$ 。当 $\lambda \geq 0$ 且 Σ 为半正定矩阵时, Hessian 半正定, 目标函数为凸函数。权重和、非负、分组和换手辅助变量约束均为线性约束, 因此可行域为凸集。对该部分模型, 局部最优点也是全局最优点, 可以直接使用 QP/LP 求解器, 而不必统一交给一般非线性优化器。

需要区分“模型是凸的”和“统计估计可靠”。协方差半正定只说明给定 Σ 下的优化结构, 不证明滚动均值能够预测未来收益。

5. L1 换手约束

原 Python 约束 $\Sigma |w_i - w_{i-1}| \leq 0.1$ 是凸约束, 但绝对值在零点不可微。High Accuracy 路径为每个资产增加辅助变量 t_i :

$$t_i \geq w_i - w_{i-1}; \quad t_i \geq -(w_i - w_{i-1}); \quad \Sigma t_i \leq 0.1.$$

这样可把绝对值约束转为线性约束, 使 LP/QP 求解器能够直接处理。求解完成后, 程序仍在原始 8 维权重空间重新计算实际 L1 换手, 避免只验证辅助模型而遗漏原约束。

6. SLSQP 原理及兼容实现

SLSQP (Sequential Least Squares Programming, 序列最小二乘规划) 属于 SQP 类带约束非线性优化算法。它在当前点附近构造目标函数的二次近似、对约束做线性化, 反复求解局部子问题并更新权重。SciPy 直接提供 SLSQP, `algo.py` 使用上一期权重作为初始点, 因此 C++ `Compat` 保留 legacy SLSQP、有限差分和同一初始权重策略。

SLSQP 可以处理非线性目标及等式、不等式约束, 但一般是局部方法。初始点、有限差分步长、容差和停止条件会影响最终点; 不同实现不保证权重逐位相同。最大分散 `ratio` 和 L1 约束的边界不可导性会放大这种差异。

7. 最大分散模型的凸化

`Compat` 直接保留原 Python 的 `ratio` 目标。High Accuracy 路径引入尺度变量 x , 固定 $v^T x = 1$, 并求解:

$$\min x^T \Sigma x, \quad \text{s.t. } v^T x = 1 \text{ 及变换后的线性约束.}$$

求得 x 后再按权重和归一化恢复 w 。该处理把固定尺度下的目标变为凸二次问题, 可使用 HiGHS QP, 并进行原始约束、对偶残差和 KKT 验收。它改变了求解路径, 不能被解释为与 Python SLSQP 逐位相同。

8. 求解器与路线选择

本项目使用 Eigen 进行固定 8 维矩阵运算，FreeXL 读取 BIFF .xls，legacy SLSQP 保持 Python 兼容，NLOPT SLSQP 作为部分失败场景的降级路径，HiGHS 处理 High Accuracy 的 LP/QP。

组件	承担的职责	采用原因
Eigen	8 维向量、协方差和约束计算	固定维度、连续存储，避免通用容器开销
HiGHS	线性规划和凸二次规划	覆盖最大收益、均值-方差和凸化最大分散
NLOPT / SLSQP	非线性兼容或降级求解	保留原模型行为并提供回退
FreeXL	读取 XLS 工作表	匹配题目文件格式，避免引入完整 Excel 运行时

未采用 IPOPT 作为主方案，因为本题主要模型可表示为 LP/QP，引入一般非线性内点求解器会增加部署复杂度。OSQP、qpOASES 和手写主动集法均有合理使用场景，但本项目未进行同机实测，因此不据此判断其速度优劣。

9. Python 与 C++ 的执行差异和性能瓶颈

Python 的耗时不等于 Python 字节码耗时。NumPy、Pandas 和 SciPy 的数值核心已进入 C/Fortran/native code，但 Python 层仍存在解释器调度、动态对象、DataFrame 临时对象、iloc 循环、rolling/cov 重复分配以及优化器 callback 开销。8 维矩阵乘法本身不是本题的主要瓶颈。

阶段	Python 主要开销	C++ 主要开销
启动与输入	解释器和 pandas 导入、read_excel	进程创建、DLL 装载、FreeXL 解析
预处理与统计	iloc、rolling、ewm/cov 的临时对象	Eigen 小矩阵和窗口构造
求解	每个调仓日 SLSQP、有限差分及 callback	Compat legacy SLSQP；High Accuracy HiGHS LP/QP
输出与退出	DataFrame 格式化、解释器回收	标准输出和第三方 DLL 清理

本实现采用固定 8 维 Eigen 类型、滚动统计缓存、六种策略共享矩估计、Release x64，并避免为了小规模问题盲目增加线程。多线程创建和同步成本可能高于矩阵计算本身；未进行并行版本同机实测，因此不据此宣称并行会更快。

10. 程序运行时间

内部计时覆盖 XLS 读取、收益转换、统计、求解和组合构造，不覆盖父进程创建、动态库加载、标准输出渲染和退出清理。实际记录如下。

模式	time_total_ms	XLS read	LP solve	QP solve	SLSQP fallback	协方差正则
HighAccuracy (--high)	56.0	18.186	6.2	35.5	0	0.25
Compat (--compat)	43.456	报告记录	不适用	不适用	SLSQP 主路径	0

完整进程基准采用 Process.Start 到 WaitForExit 的端到端定义，AMD Ryzen 5 7535U、Windows、MSVC Release x64：

模式	首次运行 ms	重复中位数 ms	P95 ms	重复最大 ms	重复最小 ms
HighAccuracy (--high)	543.00	401.23	462.76	483.34	361.60
Compat (--compat)	370.26	350.57	451.00	466.03	332.86

严格重启后的单次进程耗时为 High 403.088 ms、Compat 940.586 ms，退出码均为 0。该结果满足 1 秒目标，但 Compat 冷启动接近 1

秒，说明系统抖动可能大于算法本身。计划任务等待和系统引导时间不计入上述进程计时。

11. 与 algo.py 的结果对比

Python 基准采用项目保存的 reference.txt 快照，C++ 采用当前 MSVC Release 的 --compat 输出。误差向量顺序为年化 PnL、年化波动、Sharpe、Calmar、胜率、最大回撤；前两项、胜率和回撤单位为百分点，Sharpe/Calmar 为绝对值。

模型	Python 基准	C++ Compat	绝对误差向量	最大误差
最大分散/等权	16.54,3.44,4.814,55.534,47.30,0.41	17.90,3.46,5.177,50.003,46.85,0.50	1.36,0.02,0.363,5.531,0.45,0.09	5.531
最大分散/EWM	19.01,3.91,4.857,51.691,47.98,0.51	19.07,3.92,4.866,52.135,48.20,0.51	0.06,0.01,0.009,0.444,0.22,0.00	0.444
最大收益/普通	21.05,3.84,5.481,36.319,49.08,0.79	21.05,3.84,5.481,36.321,49.08,0.79	0.00,0.00,0.000,0.002,0.00,0.00	0.002
最大收益/EWM	21.87,3.91,5.592,39.262,50.00,0.76	21.87,3.91,5.592,39.262,50.00,0.76	0.00,0.00,0.000,0.000,0.00,0.00	0.000
风险厌恶20/普通	21.41,3.94,5.436,38.041,49.08,0.77	21.42,3.94,5.437,38.052,49.08,0.77	0.01,0.00,0.001,0.011,0.00,0.00	0.011
风险厌恶20/EWM	21.77,3.92,5.551,36.375,49.54,0.81	21.77,3.92,5.551,36.375,49.54,0.81	0.00,0.00,0.000,0.000,0.00,0.00	0.000

最大分散/等权差异最大，回撤日期从 Python 的 20210722–20210727 变为 C++ 的 20210722–20210728。项目没有保存逐日 Python/C++ 权重序列，因此未计算 $\max |w_{\text{cpp}} - w_{\text{python}}|$ 和 L1 权重差异，不据此判断权重路径孰优。

12. 结果差异原因

同一模型的数值求解差异来自不同 SLSQP 实现、有限差分步长、收敛容差、停止条件、浮点运算顺序、EWM 协方差细节和标准差自由度约定。最大分散 ratio 的平方根、分式和换手绝对值在边界处会产生平坦或不可导区域，多个可行局部终点均可能出现。

High Accuracy 的差异属于另一类问题。HAC/Newey-West、多期缩放、OAS 和 PSD 修复改变了统计估计；HiGHS QP 也改变了求解过程。因此 High Accuracy 与 algo.py 的差异不能归入兼容误差，也不能用其更小的 KKT 残差证明未来收益更高。

13. 数值稳定性与解验收

有限样本和强相关策略可能导致协方差近奇异，浮点误差还可能产生极小负特征值。High Accuracy 路径执行对称化、OAS 收缩和特征值 floor，使矩阵进入 PSD 区域并改善条件数。该处理提高给定模型的数值稳定性，但改变了统计估计，Compat 不执行这些改进。求解器返回 optimal 只是必要条件。程序还重新检查权重有限性、 $\sum(w)=1$ 、非负约束、分组约束、L1 换手、目标有限性、原始残差、对偶残差、互补性和 KKT 残差。High Accuracy 使用 $1e-9$ 级约束验收。本项目记录的 HighAccuracy 最大 KKT 残差为 $7.8e-16$ ；这只说明当前解满足给定数学模型，不能证明输入统计量准确或未来收益更优。

14. 工程结构与可维护性

输入层由 io、data 和 dates 负责 XLS、数值闸门和 Excel 日期体系；preprocessing、moments 和 moment_cache 负责日增量、 3σ 处理及滚动统计；constraints 和 optimization_model 构造统一约束模型；highs_solver、slsqp_solver、compatibility 和 solver 隔离求解器；portfolio 负责调仓与组合收益；evaluation、evaluate_pnl 和 reporting 负责指标和输出。

求解器不读取 Excel，统计模块不依赖具体求解器，评价模块不依赖 FreeXL，公共接口使用统一 Options、SolveInfo 和 PortfolioResult。新增输入格式只影响输入层，新增求解器可复用模型与验收结构。第三方类型主要留在实现模块，降低替换和测试成本。

15. 测试与回归

测试	验证内容	结果
numerical_regression	统计、约束、日期、EvaluatePNL 和求解模块回归	通过
xls_runtime	FreeXL 读取真实 data.xls 并正常运行	通过
output_high	--high 输出与 High fixture 对比	通过
output_compat	--compat 输出与 Python 兼容 fixture 对比	通过

最新 CTest 结果为 4/4 通过。默认无参数输出与 --compat 去除 time_* 行后完全一致。测试同时覆盖输出格式和运行时 DLL，而不仅是“程序能启动”。

16. 实施中的关键问题

XLS 与 DLL：FreeXL 运行时还依赖 libexpat、minizip、iconv 和 zlib，只复制 freexl-1.dll 不足以保证启动。日期体系：程序处理 Excel 1900/1904 日期模式，但跨文件比较仍需确认输入含义。SLSQP 复现：不同实现无法保证最后几位一致，因此采用兼容路径而非强行替换。换手与 ratio：通过 High Accuracy 的辅助变量和凸化分别处理，但不改变 Compat 行为。

完整耗时与内部耗时：进程启动、DLL 装载和退出清理会显著影响小数据端到端耗时，不能用 time_total_ms 代替完整运行时间。High Accuracy 与 Compat：两条路径的统计模型和求解器不同，必须分别验收，不能混用 fixture。

17. 已知隐患与边界

- Compat SLSQP 的最终权重和最大分散指标对初始点、差分步长和停止条件敏感。
- 协方差 near-singular 时，High Accuracy 的正则化改变模型；Compat 仍保留原统计行为。
- DropRow 删除缺失行后可能跨越多个自然日直接差分，严格日频场景应拒绝缺失行并单独补齐。
- 求解失败时沿用 previous weight 可保持可行性，但可能隐藏暂时停止调仓；生产使用应监控 failure、fallback 和连续失败次数。
- 当前实现固定 8 个资产，部分窗口、换手和本金参数仍需通过 Options 配置。
- 冷启动受 Windows 调度、磁盘缓存、杀毒扫描和 DLL 装载影响；Compat 单次冷启动为 940.586 ms，未进行更多冷启动样本，因此不据此判断两种算法的稳定性能排名。
- 第三方库版本、运行时 DLL 和许可证需要在交付环境固定。

18. 最终结论

本项目完成了 algo.py 时间序列组合优化流程的 C++20/MSVC

实现，并将复现与改进拆成独立路径。默认入口为 PythonCompatible，能够保持原始统计口径、legacy SLSQP 调用和输出格式；均值-方差四组策略与 Python

基准在当前显示精度内基本一致，最大分散/等权仍有明确差异，原因主要是 ratio 目标、L1 不可导边界和不同 SLSQP 数值行为。

完整进程重复运行满足 1 秒限制，Compat 中位数为 350.57 ms，HighAccuracy 中位数为 401.23

ms；严格重启后的两次单样本也小于 1 秒。High Accuracy 通过协方差修复、最大分散凸化、HiGHS LP/QP 和 KKT 验收提高给定模型的可验证性，但它改变统计模型和求解路径，只作为增强/实验方案，不作为 Python 兼容结果的替代。

在未保存逐日 Python 权重序列、未重复安装 Python

依赖进行独立重跑的前提下，本报告只对已有基准快照和实际 C++

测试作出结论；不把未实测的算法、并行方案或未来收益表现写成已验证事实。

附录 A：运行命令与交付文件

MSVC 构建：cmake_build.bat。默认复现：build\mvo.exe data.xls。显式兼容：build\mvo.exe data.xls --compat。增强实验：build\mvo.exe data.xls --high。回归测试：ctest --test-dir build --output-on-failure。完整进程基准：tools\benchmark_process.ps1 -Runs 20。

文件	用途
algo.py	原始 Python 基准程序
include/mvo、src	C++20 模块和求解实现
REPORT.md	工程过程与测试记录
API.md、STRUCTURE.md、MODULES.md	接口、结构和模块说明
REUSE_REPORT.md	第三方路线调研
tests/fixtures	High/Compat 输出回归基准
tools/benchmark_process.ps1	端到端进程性能测试